1. Introduction

This report presents the implementation and evaluation of multiple machine learning models applied to the Electricity Bill Survey dataset. The dataset was collected via a Google Form from households across Karachi and captures electricity consumption patterns, appliance usage, and billing information. The primary objective is to classify electricity consumption into three categories “Normal, High, and Shock” based on percentage change in monthly units. An additional Linear Regression model is implemented as an advancement task to predict the next month's electricity units.

2. Data Cleaning, Normalization, Encoding, and Augmentation

2.1 Raw Dataset Overview:

The raw dataset contained 16 columns including a timestamp and an empty column. After initial loading, the columns were renamed to descriptive identifiers for consistency. The timestamp and empty columns were dropped, leaving 14 usable features.

2.2 Data Cleaning Steps:

- Dropped Columns: The timestamp and empty_column features were removed as they carry no predictive information. • Missing Value Imputation: ac_count missing values were filled with the column median.

  has_heater and has_ups missing values were filled with the respective column mode (most frequent value).
- Location Standardization: Raw location strings (e.g., 'Karachi - Urban', 'Main city area (Gulshan, DHA, Clifton, etc.)') were mapped to three standardized categories: Main city, Residential colony, and Muhalla or local area.
- Month Standardization: Abbreviated month names (Jan, Feb, etc.) were expanded to full names (January, February, etc.) for consistency.

2.3 Original Dataset Features:

The following table summarizes all original features retained after cleaning:

Feature	Type	Description
location	Categorical	Area of residence (Main city / Residential colony / Muhalla)
people_count	Numeric	Number of people in household
month	Categorical	Month of billing
units_latest	Numeric	Current month electricity units consumed
units_previous	Numeric	Previous month electricity units consumed
ac_hours	Numeric	Average daily hours of AC usage
fan_count	Numeric	Number of fans in household

fridge_age	Numeric	Age of refrigerator in years
light_hours	Numeric	Average daily hours of lighting
load_shedding_hours	Numeric	Daily load shedding hours
has_solar	Categorical	Whether household has solar panel (Yes/No)
ac_count	Numeric	Number of air conditioners
has_heater	Categorical	Whether household has heater (Yes/No)
has_ups	Categorical	Whether household has UPS (Yes/No)

2.4 Feature Engineering:

Several new features were derived from the original columns to improve model performance:

Engineered Feature	Definition
total_ac_load	ac_count × ac_hours
units_diff	units_latest - units_previous
consumption_change_pct	(units_diff / units_previous) × 100
season	Derived from month (Summer / Winter / Spring / Fall)
solar / heater / ups	Binary encoding of Yes/No categorical fields
pct_change	Percentage change used to label target classes
units_per_person	units_latest / people_count

2.5 Target Variable Definition:

The target variable consumption_category_target was derived from the percentage change between the latest and previous month's units (pct_change). Three classes were defined as follows:

Class	Condition (pct_change)
Normal	< 15% increase
High	15% – 40% increase

Shock	> 40% increase

2.6 Encoding:

- Label Encoding: Categorical columns (location, month, season, has_solar, has_heater, has_ups, consumption_category) were label-encoded using scikit-learn's LabelEncoder.
- Binary Encoding: has_solar, has_heater, and has_ups were additionally converted to binary integer columns (solar, heater, ups) using a lambda function.
- Target Encoding: The target variable was encoded as integers 0 (High), 1 (Normal), 2 (Shock) for use with TensorFlow and scikit-learn models.

2.7 Dataset Split and Data Leakage:

- For all the models we used 70-15-15 ratio for train, validation and test split using stratified splitting.
- Leaky features (pct_change, units_diff, consumption_change_pct, units_latest,units_per_person) were removed from all feature matrices before training to prevent data leakage.

3. Machine Learning Models and Algorithms

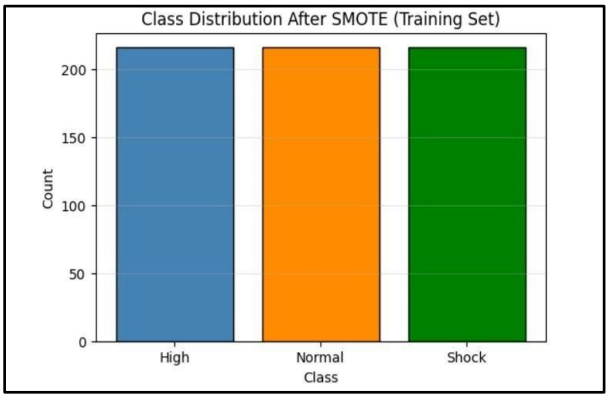
We implemented three machine learning algorithms to predict the consumption category (Normal, High, Shock). An additional regression model was implemented as an advancement to predict electricity units for the next month. The best model is decided on the basis of highest f1_score for shock class.

3.1 Artificial Neural Networks (ANN):

Three ANN models of increasing complexity were implemented using TensorFlow/Keras for multiclass classification. All models used Stochastic Gradient Descent (SGD) and sparse_categorical_crossentropy loss. The softmax activation in the output layer produces probability distributions across the three target classes.

3.1.1 Class Imbalance Handling Using SMOTE:

The dataset exhibited significant class imbalance with the Normal class being heavily over-represented. SMOTE (Synthetic Minority Over-sampling Technique) was applied exclusively on the training set with k_neighbors=3 to generate synthetic samples for the minority classes (High and Shock). Validation and test sets were left untouched to ensure unbiased evaluation.



3.1.2 Normalization using StandardScaler :

All numerical features were standardized using StandardScaler (zero mean, unit variance). The scaler was fitted only on the training data and then applied to validation and test sets to prevent data leakage.

Model 1: Simple ANN

- Architecture: Input ➐Dense(16, ReLU) ➐Dense(8, ReLU)➐Dense(3, Softmax)
- Epochs: 40, Batch Size: 32
- Characteristics: Smallest network with no regularization. Prone to underfitting due to limited capacity.

Model 2: Medium ANN

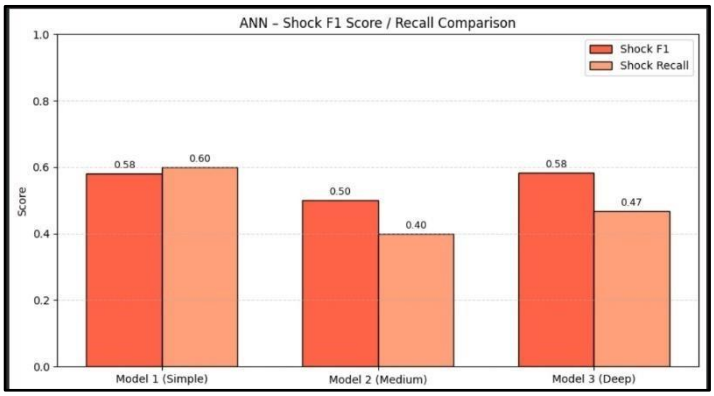
- Architecture: Input➐Dense(32, ReLU)➐Dropout(0.3)➐Dense(16, ReLU)➐Dense(3, Softmax)
- Epochs: 60, Batch Size: 32
- Characteristics: Moderate capacity with dropout regularization to reduce overfitting.

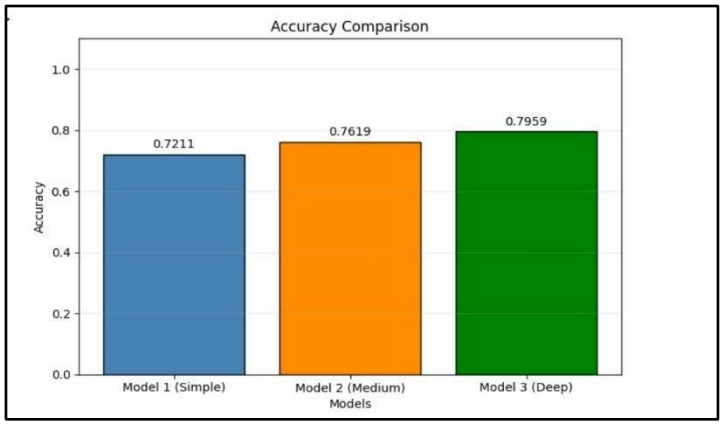
Model 3: Deep ANN

- Architecture:Input➐Dense(64,ReLU)➐Dropout(0.2)➐Dense(32,ReLU)➐Dropout(0.2) ➐Dense(16, ReLU)➐Dense(3, Softmax)
- Epochs: 80, Batch Size: 32
- Characteristics: Deepest network with staggered dropout for the best generalization.

ANN Performance Summary:

Model	Architecture	Optimizer	Epochs	Accuracy	F1 Score (Shock)
Model 1 (Simple)	Dense(16) ➐ Dense(8)	SGD lr = 0.01	40	0.72	0.58
Model 2 (Medium)	Dense(32)➐ Dropout(0.3)➐ Dense(16)	SGD lr = 0.01	60	0.76	0.50
Model 3 (Deep)	Dense(64)➐ Drop(0.2) ➐ Dense(32)➐ Drop(0.2)➐ Dense(16)	SGD lr = 0.01	80	0.80	0.58





3.2 Logistic Regression

Three Logistic Regression models were trained with varying regularization strengths (C parameter). **class_weight='balanced'** was used in all variants to handle the remaining class imbalance after SMOTE.

Model 1: Default Regularization (C = 1.0)

- C=1.0, max_iter=500, class_weight='balanced'
- Baseline model. Balances bias and variance with default L2 regularization strength.

Model 2: Strong Regularization (C = 0.01)

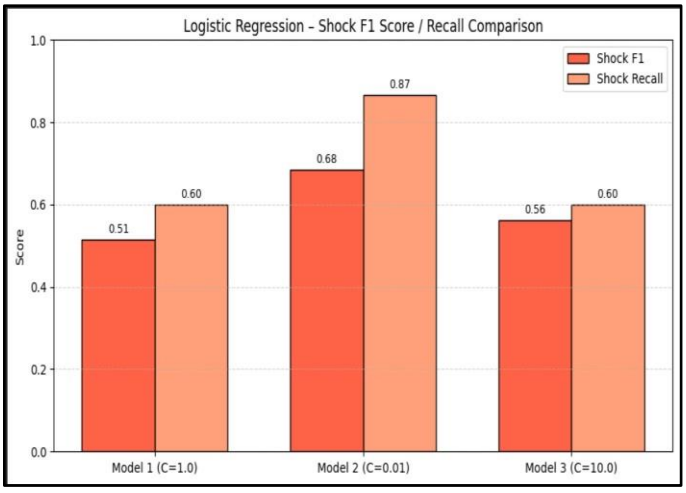
- C=0.01, max_iter=1000, class_weight='balanced'
- High penalty reduces model complexity. Useful to combat overfitting but may underfit if too aggressive.

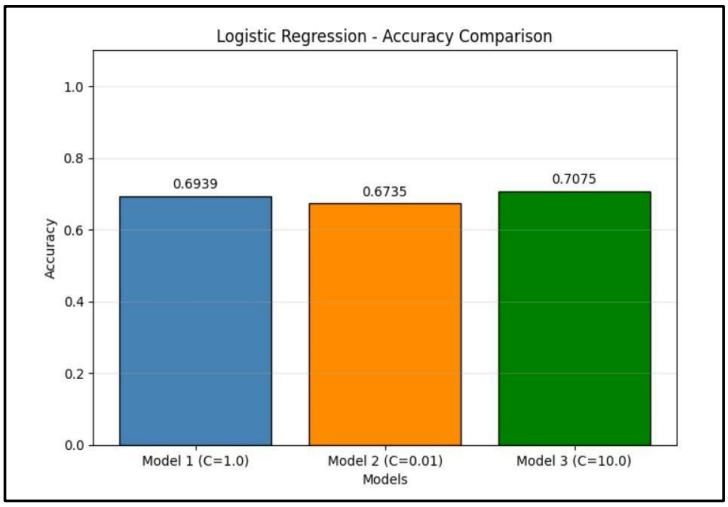
Model 3: Weak Regularization (C = 10.0)

- C=10.0, max_iter=500, class_weight='balanced'
- Low penalty allows the model more freedom, potentially capturing more patterns but risking overfitting.

Logistic Regression Performance Summary:

Model	Regularization	Accuracy	F1 Score (Shock)
Model 1	C=1.0	0.69	0.51
Model 2	C=0.01	0.67	0.68
Model 3	C=10.0	0.71	0.56





3.3 Decision Tree Classification

Three Decision Tree models were trained with different depth and splitting configurations. The Gini impurity criterion was used with **class_weight='balanced'**. Encoded location, month, and season features along with binary appliance indicators were used as inputs.

Model 1: Shallow Tree (max_depth = 3, min_samples_split = 2)

- Shallow depth severely restricts the tree from learning complex boundaries.
- High bias: training accuracy was only ~0.70, indicating underfitting.

Model 2: Balanced Tree (max_depth = 5, min_samples_split = 5)

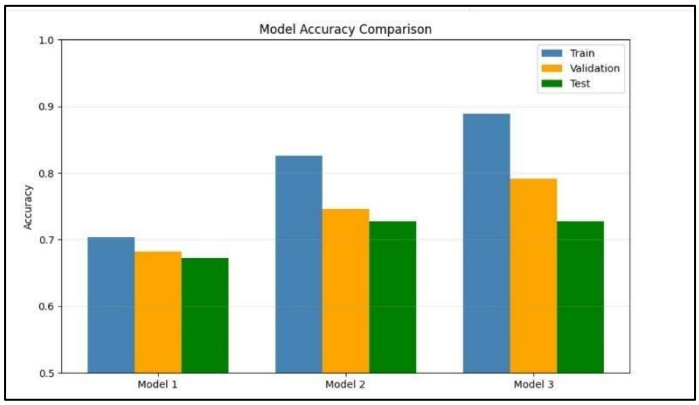
- The depth was reduced from an initial 7 to 5 after observing a train-validation gap of 0.15.
- At depth=5 the gap reduced with the same validation accuracy, indicating better generalization.
- Achieved the best F1 scores for the minority classes: High F1 = 0.29, Shock F1 = 0.53 on the validation set.

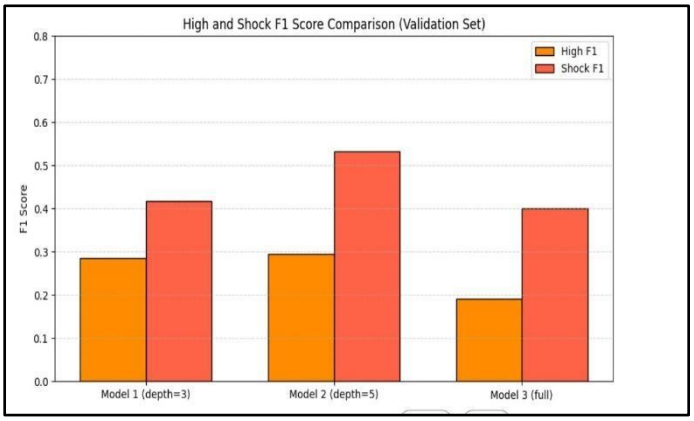
Model 3: Full Tree (max_depth = None, min_samples_split = 10)

- Grows until all leaves are pure. Training accuracy reaches ~0.95, a clear sign of overfitting.
- High F1 dropped to 0.19 on validation, meaning the High class was almost entirely missed.

Decision Tree Performance Summary:

Model	max_depth	min_samples_split	Train Acc.	Val. Acc.	Test Acc.
Model 1	3	2	0.70	0.68	0.67
Model 2	5	5	0.82	0.74	0.72
Model 3	None	10	0.88	0.79	0.72





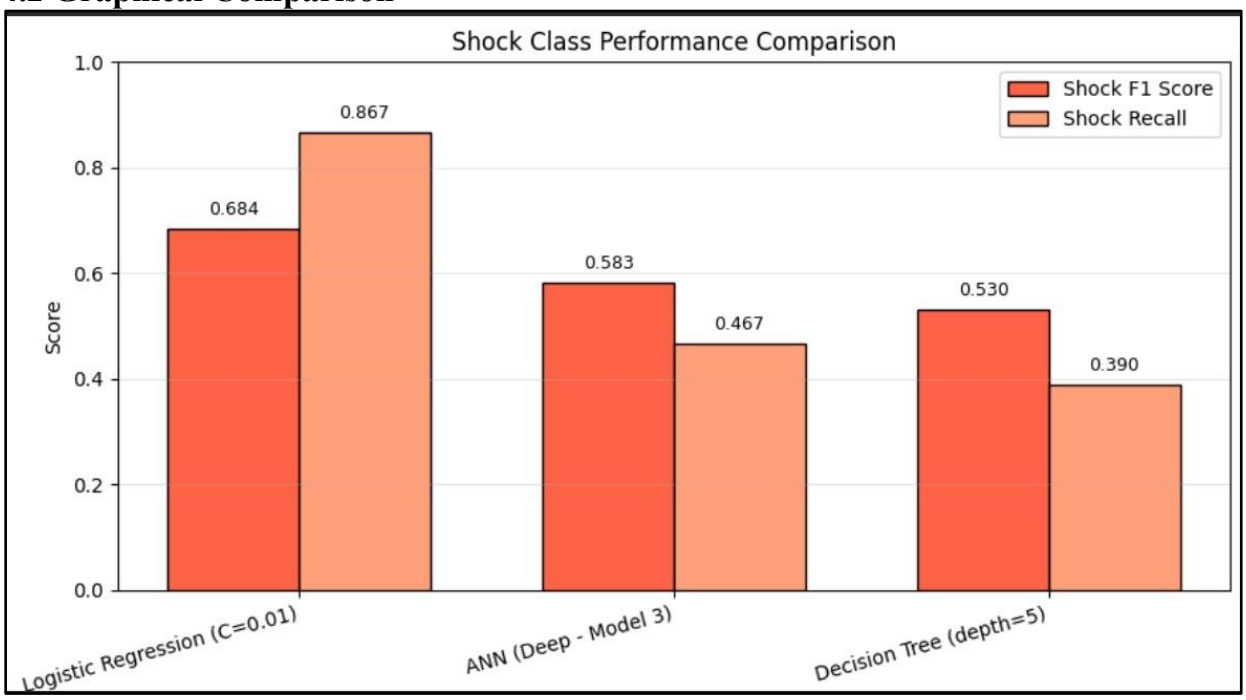
4. Model Performance Comparison

4.1 Cross-Algorithm Comparison Table

The table below summarizes the best-performing variant from each algorithm:

Algorithm	Best Variant	Accuracy	F1 Score
ANN	Model 3 (Deep)	0.80	0.58
Logistic Regression	Model 2 (C = 0.01)	0.67	0.68
Decision Tree	Model 2 (depth=5)	0.74	0.53

4.2 Graphical Comparison



Final Model:

Logistic Regression (C=0.01) is the BEST model because:

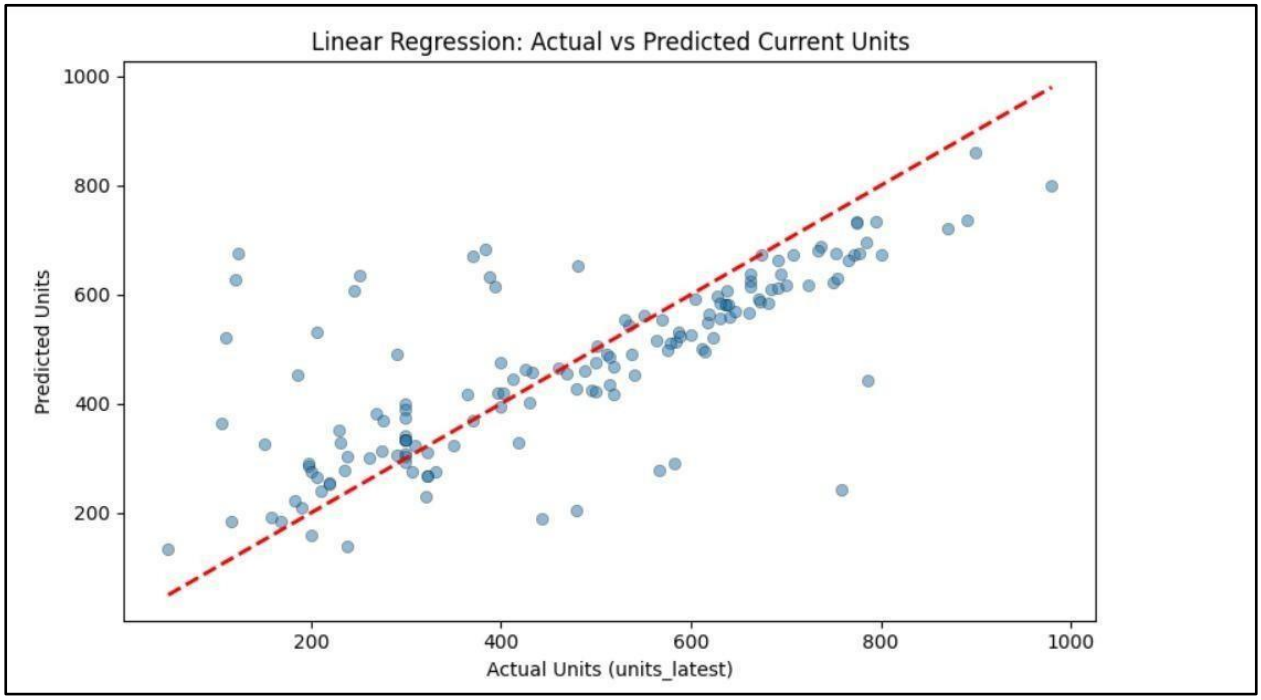
- Highest Shock F1 Score (0.6842), best at detecting abnormal bills.
- Highest Shock Recall (0.8667), catches most shock events.

5. Linear Regression (Additional Task)

As an advancement, a Linear Regression model was implemented to predict next month's electricity consumption in units. Unlike the classification models, this is a regression task. The model uses eight numerical features to predict units_latest (current month units), and a trend-projection logic in the interactive widget uses the predicted value to estimate the following month's bill.

Unique Implementation Aspect

The interactive prediction widget computes a consumption trend (latest - previous) and projects it forward before feeding input to the model. This mimics real-world forecasting where the trend line from recent bills is used to anticipate future consumption.



6. Model Performance Analysis

6.1 Underfitting

- ANN Model 1 (Simple): The model is too small (16 → 8 neurons, 40 epochs with SGD), so it fails to learn the complex non-linear patterns in the data. This leads to weaker accuracy and F1 scores compared to deeper ANN models.
- Decision Tree Model 1 (depth = 3): With very shallow depth, the model cannot properly separate overlapping classes. The training accuracy (~0.70) shows it is not even fitting the training data well.
- Logistic Regression (C = 0.01): Strong regularization makes the model overly simple, restricting its ability to capture patterns in a moderately complex dataset, which slightly reduces performance.

6.2 Overfitting

- Decision Tree Model 3 (full depth): Shows clear overfitting, with training accuracy (~0.95) much higher than validation (~0.79). The model is essentially memorizing training data instead of generalizing.
- Decision Tree tuning observation: When max_depth = 7, there was a noticeable train-validation gap (~0.15). Reducing it to depth = 5 improved generalization and reduced the gap to ~0.08 without hurting validation performance.
- ANN Model 3 (Deep): Dropout layers help reduce overfitting, but given the small dataset, there is still some risk of memorization if trained for too long.

6.2 Limitations and Future Work

Limitations:

- The dataset is relatively small and survey-based, which may limit generalization.

- The “Shock” category contains fewer real examples compared to other classes.
- The Linear Regression model still leaves room for improvement, with an R^2 score of 0.546.

Future improvements:

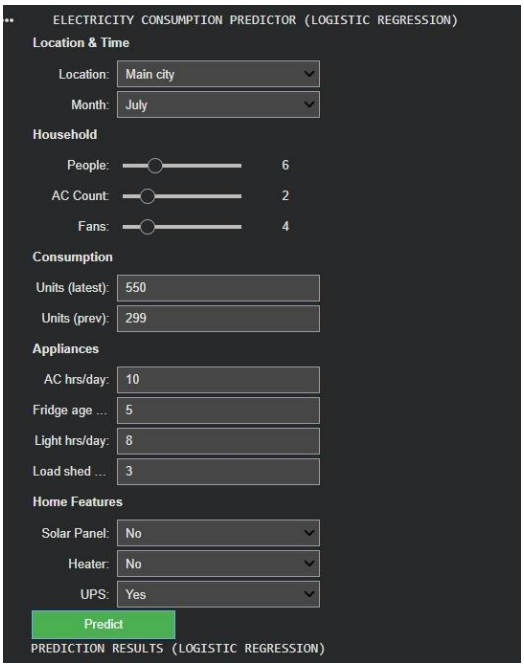
- Collecting larger and more diverse datasets.
- Using ensemble methods such as Random Forest or XGBoost.
- Applying advanced hyperparameter tuning techniques.
- Exploring cost-sensitive learning methods.
- Adding additional household and appliance-related features.

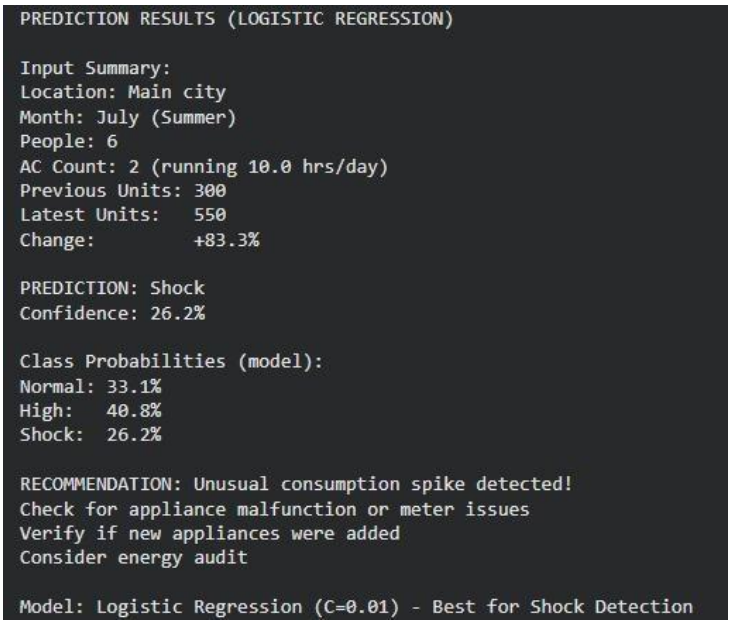
7. Test Case Runs:

Following are the test cases for the final model (logistic regression). **Test**

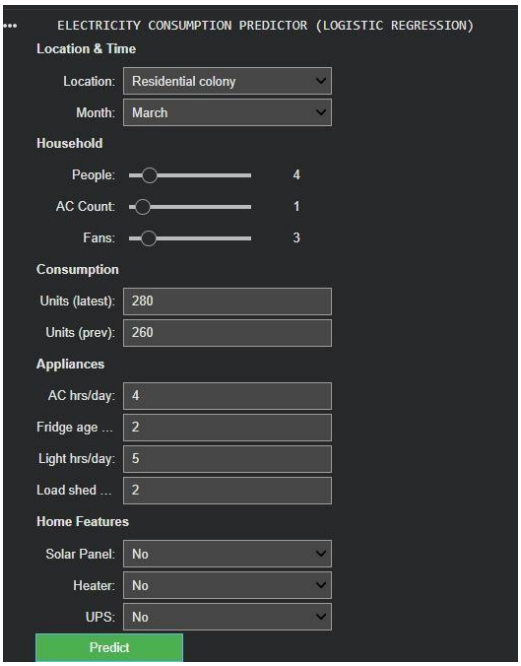
Case 1:

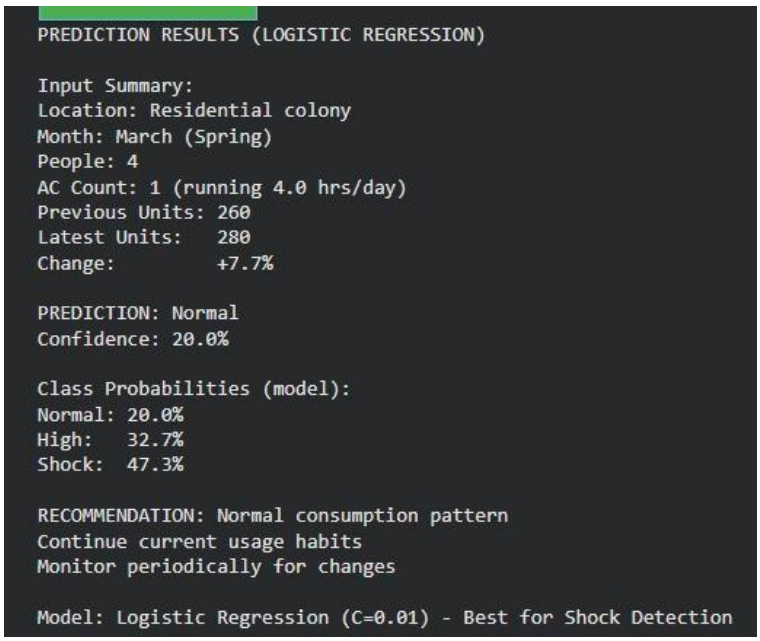
Input:



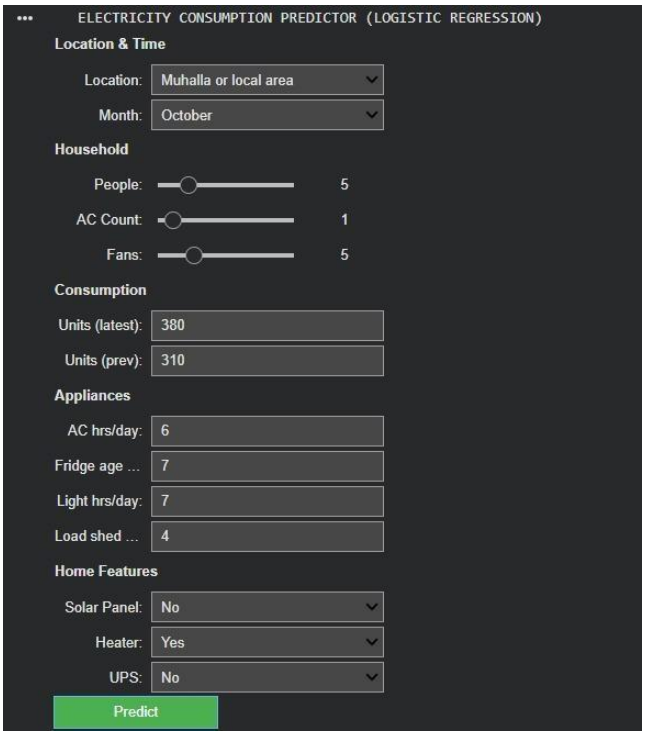


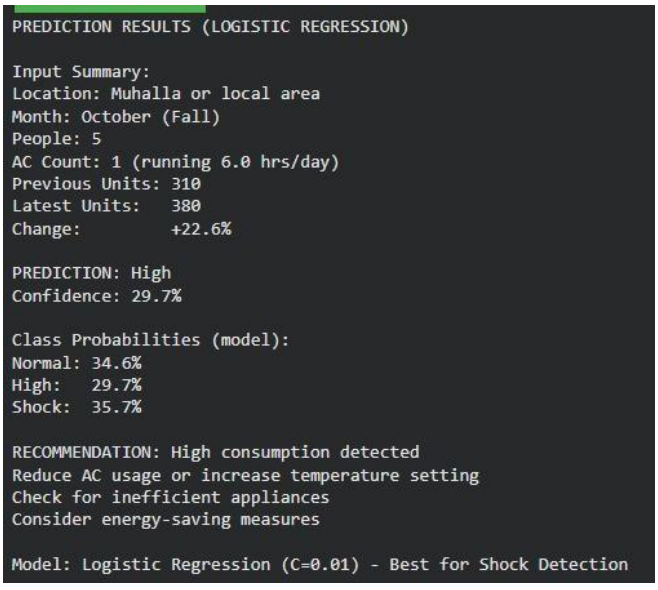
Test Case 2:





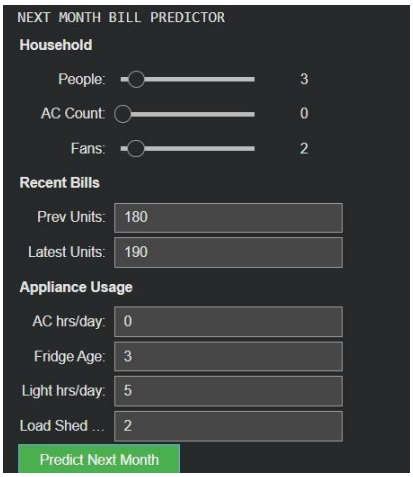
Test Case 3:

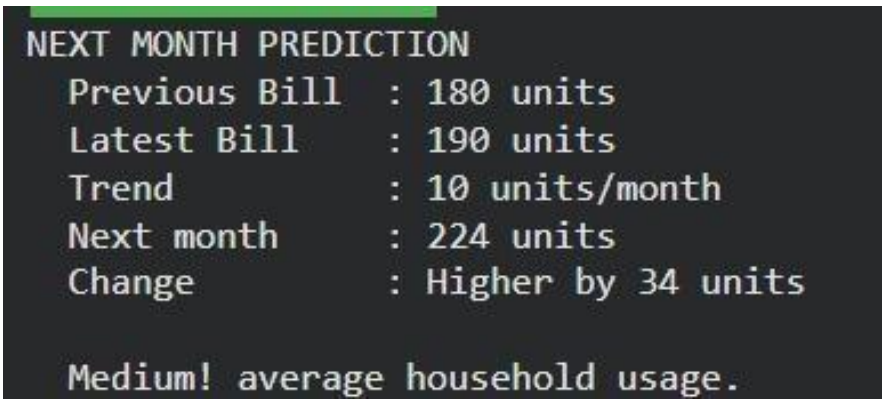




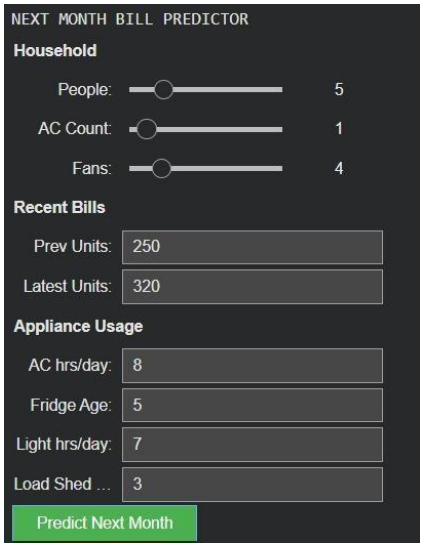
Test Case Runs for Linear Regression:

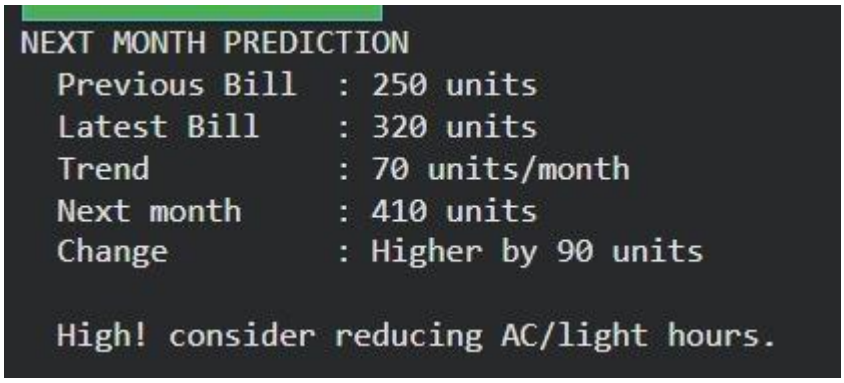
Test Case 1:



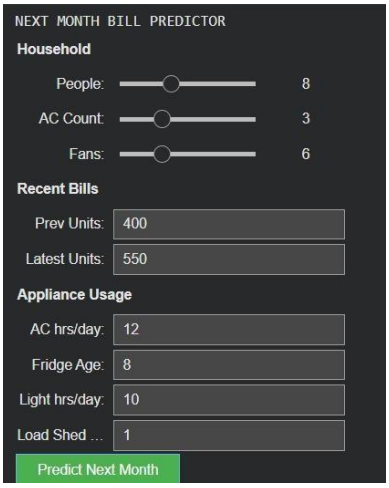


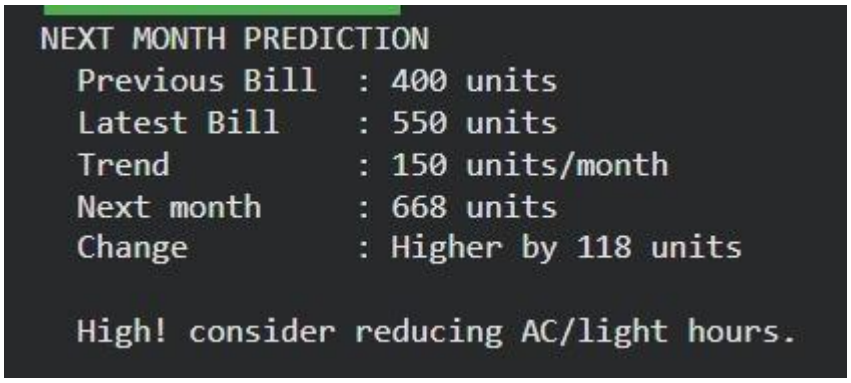
Test Case 2:





Test Case 3:





8. AI Use Declaration

Tools used: Claude and DeepSeek.

- We used for help with writing the report, organizing sections, and making explanations clearer.
- We also took help in debugging support during development of models and interactive widgets.
- No AI tool was used to generate the actual code or models. All data preprocessing, feature engineering, model training and evaluation were done solely by all group members.

Conclusion:

Using survey data gathered from Karachi households, this study created a machine learning system to categorize household electricity usage into Normal, High, and Shock categories. Several models, such as ANN, Logistic Regression, and Decision Tree, were trained and compared following data cleaning, feature engineering, and class imbalance handling with SMOTE. Because Logistic Regression performed better at identifying the crucial "Shock" situations and had acceptable interpretability, it was chosen as the final model. Furthermore, a moderately accurate Linear Regression model was developed to forecast electricity use for the following month. Overall, the system shows a useful use of machine learning for energy usage analysis and incorporates interactive widgets for real-time prediction.